

Accelerating FALCON: Speed Records for FALCON on Xilinx FPGAs

Sharath Pendyala¹, Rahul Magesh¹, Elif Bilge Kavun² and Aydin Aysu¹

¹ Department of Electrical and Computer Engineering, North Carolina State University, USA
spendya@ncsu.edu, rmagesh@ncsu.edu, aaysu@ncsu.edu

² Barkhausen Institut, Germany
elif.kavun@barkhauseninstitut.org

Abstract. FALCON is a NIST-selected post-quantum digital signature scheme whose performance bottleneck lies in the **SamplerZ** subroutine for discrete Gaussian sampling. We present a throughput-optimized, custom hardware implementation of **SamplerZ** that introduces several architectural and algorithmic innovations to significantly accelerate signature generation. Our design incorporates a datapath-aware floating-point arithmetic pipeline that strategically balances latency and resource utilization. Our novel algorithmic innovations include an Estrin’s Scheme-based polynomial evaluator and a constant-latency **BerExp** routine using floating-point exponentiation IP to eliminate fixed-point decomposition critical paths. Additionally, we optimize rejection handling through parallel sampling loops and propose a speed-optimized flooring circuit. These advancements lower the sampling time by 55%-81% and overall FALCON signature generation time by 36%-53% compared to the state-of-the-art FPGA implementation. In a landmark result, our work is the first to demonstrate a Xilinx FPGA **SamplerZ** design that outperforms state-of-the-art software (by 15%) and ASIC (by 16%) designs, advancing the practical deployment of post-quantum signatures on reconfigurable hardware.

Keywords: Post-Quantum Cryptography · FALCON · discrete Gaussian sampling · High-Performance · Configurable · FPGA

1 Introduction

Quantum computers exploit the principles of quantum superposition and entanglement to process information in fundamentally different ways compared to classical machines. Whereas classical algorithms for integer factorization such as the general number field sieve run in sub-exponential time and are infeasible against RSA [RSA78] moduli of sufficient size (e.g., 2048 bits), Shor’s algorithm [Sho97] solves the problem in polynomial time, rendering RSA and Diffie-Hellman [DH76] cryptosystems practically insecure once a sufficiently large and fault-tolerant quantum computer becomes available. Similarly, elliptic curve cryptography (ECC), which underpins many modern secure communication protocols due to its efficiency, is also vulnerable since Shor’s algorithm applies to the discrete logarithm problem over elliptic curves. The threat is not merely theoretical; major protocols like TLS and SSH rely on these hard problems for key exchange and signatures. Accordingly, the National Institute of Standards and Technology (NIST) and other global standardization bodies are standardizing post-quantum cryptography (PQC) to survive quantum attacks.

Among the post-quantum digital signature candidates, FALCON (Fast Fourier Lattice-based Compact Signatures over NTRU) [FHK⁺18] has been selected by NIST for standardization as FIPS 206 [Per25]. FALCON belongs to the class of lattice-based cryptographic schemes and operates under the hash-and-sign paradigm [Nat15], with security grounded in the hardness of the Ring Short Integer Solution (Ring-SIS) problem [Ajt96]. Notably,

among the three NIST-standardized digital signature algorithms [BDK⁺22, BHH⁺22], FALCON offers the most compact signature sizes, a critical advantage for conserving network bandwidth. While its complex floating-point arithmetic and higher resource cost may pose challenges for ultra-lightweight devices, these characteristics align well with performance-optimized platforms like FPGAs. The need for high-throughput, low-latency signature generation makes FALCON a compelling candidate for secure, high-speed communication systems where bandwidth efficiency is critical.

Among the three core operations in FALCON—key generation, signature generation, and verification, only the latter two are repeatedly executed on deployed devices. Key generation, though computationally expensive due to trapdoor sampling and lattice basis construction, is often performed once per session or even once per device lifetime and can be offloaded to a trusted setup phase. In contrast, signature generation and verification can be invoked thousands of times using the same key, making their acceleration vital for real-world deployments. While verification is relatively lightweight, involving norm bounds and simple integer arithmetic, signature generation is considerably more demanding and thus represents the primary target for hardware optimization.

Within the signature generation process, prior works [YSZ⁺24, OZZ⁺24, LYN⁺24, NG23] have identified the **SamplerZ** subroutine as the primary performance bottleneck. This subroutine, responsible for discrete Gaussian sampling, is invoked multiple times as a pair per signature and has been shown to account up to **64% signature generation cycles on hardware**[OZZ⁺24]. These characteristics make **SamplerZ** a compelling target for speed-oriented hardware acceleration—particularly in performance-critical deployment scenarios.

In this work, we accelerate the **SamplerZ** subroutine through architectural and algorithmic optimizations, including improved polynomial approximation, floating-point exponentiation, and early non-critical path computation. Our FPGA implementation reduces FALCON sampling and signature generation times by up to 81% and 53%, respectively, compared to the state-of-the-art FPGA [OZZ⁺24], while ensuring constant-time execution. Moreover, our design achieves the fastest FALCON sampling time, outperforming state-of-the-art ASIC [OZZ⁺24] and software [NG23] by 16% and 15%, respectively.

Prior Work. Recent efforts to accelerate FALCON signature generation on FPGAs have yielded varying architectural strategies. Existing works include the hardware/software co-design on Artix-7 by Karabulut *et al.* [KA24] and hybrid HLS-based designs on Zynq UltraScale+ and ASIC platforms [YSZ⁺24, SAW⁺23, LYN⁺24]. In contrast, Ouyang *et al.* [OZZ⁺24] introduce a dedicated full hardware architecture for the Zynq-Ultrascale+. Among existing implementations, several prior works explicitly isolate the **SamplerZ** subroutine in their architectural breakdowns [KA24, YSZ⁺24, OZZ⁺24]. Among these, the **TCHES 2025** implementation by Ouyang *et al.* [OZZ⁺24] achieves the best performance for hardware-accelerated Gaussian sampling in FALCON. Collectively, these works highlight the importance of **SamplerZ** in the signature generation pipeline and motivate the need for its dedicated hardware acceleration. Among software implementations of FALCON, the state of the art on high-performance CPUs is represented by the work of Nguyen *et al.*, targeting the Apple M1 platform [NG23].

Our Major Contributions. Despite this progress, opportunities remain to further reduce latency and complexity, particularly by minimizing control path overhead, improving rejection sampling efficiency, and accelerating exponential approximations. We list our contributions as follows:

- **Surpassing State-of-the-Art ASIC and Software Performance.** We present the first FPGA implementation of **SamplerZ** to outperform the leading optimized ASIC [OZZ⁺24] and software implementations [NG23] (Section 4.2).
- **Estrin’s Scheme based polynomial evaluation.** To accelerate the **ApproxExp** computation within **BerExp**, we replace the existing Horner-style sequential polynomial evaluation [Hor20] with Estrin’s Scheme [Est60], which exploits parallelism by

grouping terms with common exponent bases, allowing simultaneous evaluation of multiple subexpressions. This transformation yields 30% reduction in the execution time of **SamplerZ** over Horner’s Method, at the expense of up to 80 additional DSP slices. (Section 3.2.1).

- **Floating-point based BerExp.** We propose an alternative **BerExp** formulation that leverages a constant-latency floating-point exponential IP core. This eliminates the need for the integral approximation and intermediate decomposition stages in favor of a simplified datapath, resulting in 47% reduction in execution time of **SamplerZ** over traditional **BerExp**, while requiring 14 fewer DSP slices (Section 3.2.2).
- **Quantitative analysis on optimizing parallel sampling loops.** We perform a thorough quantitative analysis to optimize the number of parallel sampling loops for **SamplerZ** designs based on the rejection statistics for **BerExp**, providing three design solutions depending on the designer’s goal, capable of reducing the execution time of **SamplerZ** up to 60%. These results provide a more realistic and surprising insight in the context of hardware designs, especially in comparison to previous work by Lee *et al.* [LYN⁺24]. To enable a uniform comparison across diverse architectures, we introduce the rejection adjusted execution time (RAET) metric for **SamplerZ**. (Section 3.1.4).
- **Datapath-aware floating-point units (FPU).** We introduce a datapath-sensitive instantiation strategy for floating-point arithmetic units, where non-critical path operators are implemented using high-latency, resource-efficient IPs, and latency-critical components are instantiated with low-latency variants. This optimization improves timing closure and critical path slack, and improves the execution time of **SamplerZ** by 14%-22% (Section 3.1.2).
- **Parallelized Pre-Sampler and Sampling.** Our design enables concurrent execution of the **SamplerZ** Pre-Sampler and Sampling Loops. While the Pre-Sampler is invoked once per sample, the Sampling Loop may iterate several times depending on rejection. By decoupling these operations, we shorten the critical datapath and improve the latency by 7 cycles, significantly improving execution time of low latency **SamplerZ** (Section 3.1.1).
- **Speed-Optimized flooring circuit.** We design a parallelized flooring circuit for floating-point inputs, named **FPR_Floor**, tailored for **SamplerZ**. Unlike conventional floor operations that rely on serial conversions (floating-point to integer, back to floating-point, followed by subtraction), our implementation evaluates both possible floor candidates in parallel and selects the output based on sign comparison. This reduces **SamplerZ**’s latency to 3 cycles compared to the 5 cycles in the prior design by Ouyang *et al.* [OZZ⁺24], at the cost of 3 additional DSPs (Section 3.1.3).

Combined Impact. Together, these optimizations enable an efficient, scalable, and reproducible **SamplerZ** hardware suitable for deployment in high-throughput FALCON accelerators on FPGA platforms. Our designs offer measurable improvements in latency, execution time, and resource utilization while executing in constant time. All these contributions lead to 55%-81% improvement in **SamplerZ** execution time and 36%-46% improvement in full signature generation over state-of-the-art FPGA design [OZZ⁺24].

Our FPGA-based **SamplerZ** design outperforms state-of-the-art ASIC design [OZZ⁺24] (28nm TSMC) by 16% and state-of-the-art software design [NG23] (5nm Apple M1) by 15%, with this design being the first FPGA **SamplerZ** design to outperform state-of-the-art ASIC and high-performance CPU. Moreover, our estimated ASIC results at 28nm are expected to improve **SamplerZ** and full signature generation execution time by 67% and 42% over state-of-the-art ASIC design [OZZ⁺24], and 66% and 21% over state-of-the-art software [NG23], respectively.

The remainder of this paper is organized as follows: Section 2 reviews FALCON’s sampling architecture, its algorithms, and the role of **SamplerZ**. Section 3 outlines our

architectural, circuit-level, and algorithmic innovations and implementation details. Section 4 analyzes performance against other designs, whereas Section 5 discusses related aspects, providing valuable insights for designers. We conclude in Section 6 with future research directions.

2 Background

Several hardware and software implementations of FALCON [OZZ⁺24, KA24, QA25, KSS22, LYN⁺24, SAW⁺23, YSZ⁺24, NG23, SBN⁺20] identify the recursive Fast Fourier Sampling (FFSampling) routine as a primary computational bottleneck in signature generation. At the core of FFSampling lies the **SamplerZ** subroutine, which performs discrete Gaussian sampling using a rejection sampling scheme, posing non-trivial challenges for efficient hardware realization. As noted by Ouyang *et al.* [OZZ⁺24], **SamplerZ** alone accounts for approximately 64% of the total signature generation time on hardware, making it a critical target for hardware acceleration. Within the broader FFSampling pipeline, **SamplerZ** dominates execution time due to its repeated invocation across recursive tree levels. The rest of this section provides an overview of the FALCON signature scheme and its key computational bottleneck **SamplerZ**.

2.1 FALCON Signature Scheme

Digital signature schemes are fundamental cryptographic primitives that provide authenticity, integrity, and non-repudiation in digital communications. They are widely deployed in secure protocols such as SSL/TLS to prevent message forgery and tampering. FALCON (Fast Fourier Lattice-based Compact Signatures over NTRU) [FHK⁺18] is a lattice-based digital signature scheme designed to offer post-quantum security while maintaining compact signatures and high efficiency. It is built on the NTRU lattice structure [HPS98] and leverages Fast Fourier Transform (FFT)-based operations [DP16] to accelerate arithmetic in polynomial rings.

The scheme operates in three phases:

1. **Key Generation.** Generates a public key and a trapdoor-based private key using NTRU lattice parameters. The trapdoor allows for efficient and secure sampling from discrete Gaussian distributions.
2. **Signature Generation.** Signs a message by hashing it to a lattice point and then solving a Closest Vector Problem (CVP) using recursive Gaussian sampling.
3. **Signature Verification.** Checks the validity of the signature by ensuring its norm is below a security-bound threshold and that it maps correctly to the message hash.

FALCON has been selected by NIST as one of three finalist digital signature schemes for standardization [CSD17]. It achieves quantum resistance based on the hardness of the Ring-SIS problem [Ajt96], and stands out for its compact signature sizes, making it suitable for bandwidth- and memory-constrained applications.

Our Implementation. Our design supports both FALCON-512 and FALCON-1024 signature generation by using a configurable minimum standard deviation ($\sigma_{\min}$). By contrast, RVCE-FAL [YSZ⁺24] targets a single variant of FALCON at a time by hardcoding its Gaussian parameters, limiting generality.

Gaussian Sampling. At the core of FALCON’s signature generation lies the requirement for efficient and high-precision discrete Gaussian sampling. FALCON achieves this through the recursive FFSampling algorithm [DP16], which repeatedly invokes a discrete Gaussian sampler, **SamplerZ**, that constitutes the primary computational bottleneck.

In the following sections, we focus on explaining **SamplerZ** and its subroutines.

2.2 SamplerZ

The **SamplerZ** routine produces an integer from a discrete Gaussian distribution [HKR⁺18, KLL21] with center μ and deviation σ' . Algorithm 1 shows **SamplerZ**. The function

UniformBits refers to a source of uniform randomness, which is used throughout the algorithm for sampling and rejection decisions.

Algorithm 1 $\text{SamplerZ}(\mu, \sigma')$

Require: $\mu \in \mathbb{R}, \sigma' \in [\sigma_{\min}, \sigma_{\max}]$

Ensure: $z \sim D_{\mathbb{Z}, \mu, \sigma'}$

```

1:  $r \leftarrow \mu - \lfloor \mu \rfloor$ 
2:  $\text{ccs} \leftarrow \sigma_{\min} / \sigma'$ 
3: while true do
4:    $z_0 \leftarrow \text{BaseSampler}()$ 
5:    $b \leftarrow \text{UniformBits}(8) \& 0x1$ 
6:    $z \leftarrow b + (2 \cdot b - 1) \cdot z_0$ 
7:    $x \leftarrow \frac{(z-r)^2}{2 \cdot \sigma'^2} - \frac{z_0^2}{2 \cdot \sigma_{\max}^2}$ 
8:   if  $\text{BerExp}(x, \text{ccs}) = 1$  then
9:     return  $z + \lfloor \mu \rfloor$ 
10:  end if
11: end while

```

The algorithm first samples an integer z_0 from a fixed half-Gaussian distribution with standard deviation $\sigma_{\max} = 1.8205$ (line 4), using the **BaseSampler**. A rejection step then evaluates whether to accept the sample. It computes an approximation of the acceptance probability $\text{ccs} \cdot e^{-x}$ (lines 7-8), where ccs is a scaling factor $\sigma_{\min} / \sigma'$, and x is derived from a centered squared distance penalty. The exponential term is evaluated in a time-constant form using a loop over precomputed constants C [ZSL20]. Finally, the sample's validity is checked using a byte-wise rejection loop embedded in the call to **BerExp** (line 8).

2.2.1 Base Sampler

The **BaseSampler** subroutine (Algorithm 2) performs constant-time discrete Gaussian sampling over non-negative integers using a precomputed Reverse Cumulative Distribution Table (RCDT). The RCDT stores scaled probabilities $\Pr[z \geq i]$ for a distribution with standard deviation $\sigma_{\max}$. To sample $z \in \{0, \dots, 18\}$, the algorithm compares 72 bits of uniform randomness (u) against the table entries in a sequential loop. The output z_0 is determined by the number of thresholds u exceeds, effectively implementing inversion sampling while maintaining constant-time execution to prevent timing side-channels.

Algorithm 2 BaseSampler

Require: RCDT table $\text{RCDT}[0 \dots 18]$, uniform random bit source

Ensure: Sample $z \sim \chi$ where χ is defined by the discrete Gaussian RCDT

```

1:  $u \leftarrow \text{UniformBits}(72)$ 
2:  $z_0 \leftarrow 0$ 
3: for  $i \leftarrow 0$  to 17 do
4:    $z_0 \leftarrow z_0 + [u < \text{RCDT}[i]]$ 
5: end for
6: return  $z_0$ 

```

2.2.2 ApproxExp

The **ApproxExp** subroutine in FALCON approximates the exponential function $\exp(-x)$ using a truncated polynomial of the form:

$$P(x) = \sum_{i=0}^{n=12} a_i x^i \quad (1)$$

where the coefficients a_i are precomputed constants selected to approximate $\exp(-x)$ over a bounded domain, typically $x \in [0, \ln 2]$. This polynomial approximation is derived from a Taylor expansion centered at zero:

$$\exp(-x) = 1 - x + \frac{x^2}{2!} - \frac{x^3}{3!} + \cdots + \frac{(-x)^n}{n!} + \cdots \quad (2)$$

Truncating this expansion to a fixed degree n allows for a predictable number of operations as shown in Algorithm 3, and the alternating signs are absorbed into the coefficients a_i for evaluation.

Algorithm 3 ApproxExp(x, ccs)

Require: $x \in [0, \ln 2]$, $ccs \in [0, 1]$

Ensure: $y \approx 2^{63} \cdot ccs \cdot \exp(-x)$ (integer)

- 1: $C \leftarrow$ **Precomputed Table of 13 Constants**
 - 2: $y \leftarrow C[0]$
 - 3: $z \leftarrow \lfloor 2^{63} \cdot x \rfloor$
 - 4: **for** $i \leftarrow 1$ to 12 **do**
 - 5: $y \leftarrow C[i] - ((z \cdot y) \gg 63)$
 - 6: **end for**
 - 7: $z \leftarrow \lfloor 2^{63} \cdot ccs \rfloor$
 - 8: $y \leftarrow (z \cdot y) \gg 63$
 - 9: **return** y
-

Horner’s Method. To evaluate the polynomial efficiently in hardware or software, FALCON employs Horner’s method, which rewrites the polynomial into a nested form:

$$P(x) = a_0 + x(a_1 + x(a_2 + x(a_3 + \cdots + xa_n) \cdots)) \quad (3)$$

This formulation minimizes the number of required multiplications and additions to n each, and only requires a single register to accumulate the result. Its control simplicity and low resource footprint make Horner’s rule well-suited for compact hardware circuits. However, Horner’s scheme is inherently sequential, as each multiplication depends on the result of the previous one. This limits throughput in high-performance implementations.

Motivation for Integer Polynomial Approximation. The use of fixed-point polynomial approximations [ZSL20] in FALCON, rather than native floating-point exponentiation instructions (e.g., `exp()` or `vexpq`), stems from security considerations. On most modern CPUs, including x86 and ARM architectures, exponentiation is implemented using microcode that exhibits data-dependent timing. This variability introduces a potential side-channel leakage vector, as execution time may correlate with the magnitude of the input. By contrast, evaluating a degree- n polynomial in fixed-point arithmetic ensures that **ApproxExp** runs in constant time, regardless of the input value. This is essential for preventing timing-based attacks in signature generation.

At the time of writing, all known FALCON implementations, including those in the reference software [DP16], state-of-the-art software [NG23], and state-of-the-art hardware FalconSign [OZZ⁺24], use Horner’s scheme exclusively for evaluating **ApproxExp**.

2.2.3 BerExp

The **BerExp** subroutine shown in Algorithm 4, implements a Bernoulli accept/reject test that returns 1 with probability

$$p = \text{ccs} \cdot \exp(-x), \quad (4)$$

where $x \in \mathbb{R}_{\geq 0}$ and $\text{ccs} \in [0, 1]$. This probabilistic test is a key step in the rejection sampling process used by FALCON's **SamplerZ** routine to enforce correct discrete Gaussian distribution. Since direct evaluation of $\exp(-x)$ is both computationally expensive and potentially vulnerable to timing side-channels, **BerExp** reformulates this acceptance probability into a fixed-point integer comparison, which can be executed in constant-time.

Algorithm 4 **BerExp**(x, ccs)

Require: Floating-point values $x, \text{ccs} \geq 0$

Ensure: A single bit, equal to 1 with probability $\approx \text{ccs} \cdot \exp(-x)$

```

1:  $r \leftarrow x - s \cdot \ln(2)$ 
2:  $s \leftarrow \min(s, 63)$ 
3:  $z \leftarrow (2 \cdot \text{ApproxExp}(r, \text{ccs}) - 1) \gg s \triangleright z \approx 2^{64-s} \cdot \text{ccs} \cdot \exp(-r) = 2^{64} \cdot \text{ccs} \cdot \exp(-x)$ 
4:  $i \leftarrow 64$ 
5: repeat
6:    $i \leftarrow i - 8$ 
7:    $w \leftarrow \text{UniformBits}(8) - ((z \gg i) \& 0xFF)$ 
8: until  $(w \neq 0) \vee (i \leq 0) \triangleright$  This loop does not need to be done in constant-time
9: return  $[w < 0] \triangleright$  Return 1 with probability  $2^{-64} \cdot z \approx \text{ccs} \cdot \exp(-x)$ 
```

The procedure begins by decomposing the input x as $x = s \cdot \ln 2 + r$, where $s = \lfloor x / \ln 2 \rfloor$ is the integer shift and $r = x - s \cdot \ln 2$ is the bounded remainder such that $r \in [0, \ln 2]$. This enables the identity $\exp(-x) = 2^{-s} \cdot \exp(-r)$, allowing the exponentiation to be split into a base-2 scaling and a bounded-range exponential.

The bounded exponential term $\exp(-r)$ is then approximated using the **ApproxExp** routine, which returns a 64-bit fixed-point integer approximating $2^{63} \cdot \text{ccs} \cdot \exp(-r)$. This value is subsequently doubled, decremented by 1, and right-shifted by s , yielding

$$z = \frac{2 \cdot \text{ApproxExp}(r, \text{ccs}) - 1}{2^s} \approx 2^{64-s} \cdot \text{ccs} \cdot \exp(-r) = 2^{64} \cdot \text{ccs} \cdot \exp(-x).$$

The arithmetic shift ensures proper rounding behavior, and the resulting value z is a 64-bit integer used for probabilistic comparison.

A rejection sampling loop is then used to compare sampled uniform random bytes against the high-order bits of z , one byte at a time. In each iteration, a byte w is computed as

$$w \leftarrow \text{UniformBits}(8) - ((z \gg i) \& 0xFF),$$

where i is decremented by 8 on each round. If $w \neq 0$, the comparison terminates early. The loop halts if either $w \neq 0$ or $i \leq 0$. The result is accepted if $w < 0$, and rejected otherwise. This ensures that the subroutine returns 1 with probability approximately equal to $\text{ccs} \cdot \exp(-x)$. By performing exponentiation in the fixed-point domain and avoiding floating-point division, **BerExp** achieves constant-time execution with respect to the input x , except for the byte-by-byte comparison loop.

3 Hardware Design

This section introduces several key innovations that collectively enable a high-throughput implementation of the **SamplerZ** subroutine for FALCON post-quantum cryptography on

FPGA platforms. These innovations are categorized into two broad types: **architectural and circuit-level** techniques, and **algorithmic** optimizations. We present configurable variants to accommodate varying design objectives such as minimizing the execution time, optimizing for resource utilization, or balancing both. These contributions form the foundation of our **SamplerZ** accelerator and differentiate it from prior work in both methodology and performance.

3.1 Architectural and Circuit-Level Innovations

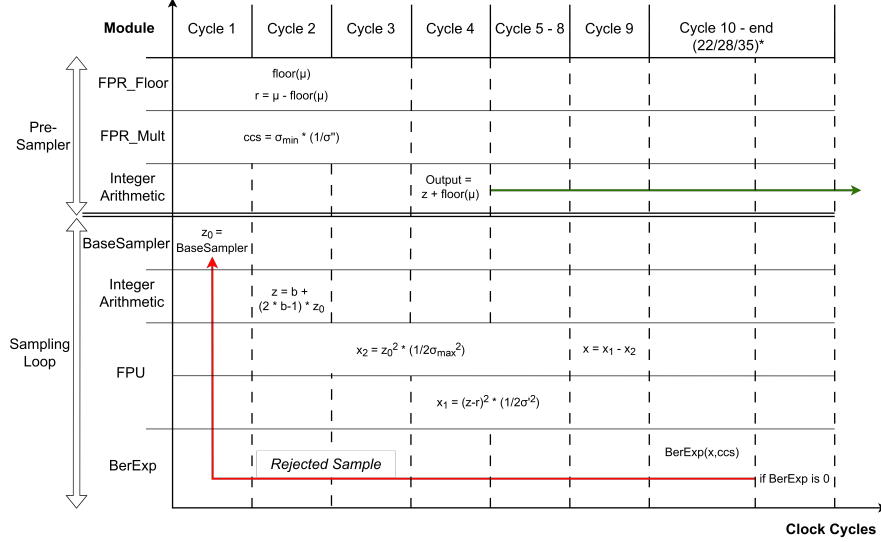


Figure 1: Scheduling diagram of our **SamplerZ**. We propose three distinct designs that take 22, 28, and 35 cycles, respectively.

3.1.1 Parallelization of Pre-Sampler and Sampling Loop

Figure 1 illustrates the cycle-accurate schedule of our **SamplerZ** implementation. The **Pre-Sampler** corresponds to lines 1-2 of Algorithm 1, which use σ' and μ to compute ccs , $\text{floor}(\mu)$, and $r = \mu - \text{floor}(\mu)$ via our custom, speed-optimized **FPR_Floor** module. The **Sampling Loop** corresponds to lines 3-11 of Algorithm 1. We exploit fine-grained parallelism at the circuit level by decoupling the Pre-Sampler from the Sampling Loop. In this design, the Pre-Sampler is executed once per invocation, while the Sampling Loop may iterate multiple times depending on the sampled values.

While prior work [OZZ⁺24] employs a similar Pre-Sampler and Sampling Loop strategy, we further optimize this architecture, as shown in Figure 2, by advancing non-data-dependent operations ahead of the Pre-Sampler stage, yielding additional latency savings. Specifically, the **BaseSampler** and the computation of b and z proceed in parallel with **FPR_Floor** and ccs generation. Once r becomes available, the FPU immediately begins evaluating the acceptance condition x . The output $z + \text{floor}(\mu)$ is continuously computed, but its associated valid signal is gated by the result of **BerExp**. This hardware-level parallelization strategy yields a net reduction in total latency and is visualized in the scheduling diagram in Figure 1.

This parallelization yields a net latency reduction of 7 cycles (visualized in Figure 1), comprising: (1) a 3-cycle improvement from our optimized **FPR_Floor** datapath over the state-of-the-art, as detailed in Section 3.1.3; (2) a 3-cycle gain from overlapping the input stage of the Pre-Sampler with the Sampling Loop; and (3) a 1-cycle saving by precomputing the output $z + \text{floor}(\mu)$ ahead of **BerExp** evaluation.

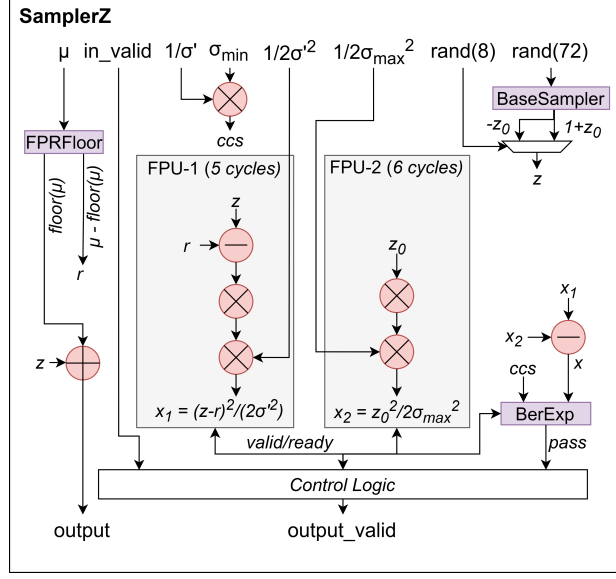


Figure 2: High level block diagram of our **SamplerZ**. The figure omits register stages for visual simplicity. Individual modules are expanded under their subsections.

3.1.2 Datapath-aware floating-point IP instantiation

An investigation of the **SamplerZ** datapath reveals an opportunity to optimize performance by migrating a subset of floating-point operations from the critical path. Based on this, we introduce architectural optimizations at the datapath level by differentiating the instantiation of FPUs based on their critical path within the datapath. A key example is the computation of the variable x in the sampling loop (line 7 of Algorithm 1). Although the final value of x lies on the critical path, since the **BerExp** operation cannot begin until x is computed, the sub-expression $z_0^2 / (2 \cdot \sigma_{\max}^2)$ itself is not latency-critical. When the pre-sampler and sampling loops are executed in parallel, this sub-expression can tolerate a latency overhead of up to 3 clock cycles without affecting the overall critical path. In scenarios where the Sampling Loop is re-executed due to sample rejection, this tolerance margin reduces to 2 clock cycles. Consequently, the **FPR_Mult** used for this operation can have increased latency of at least 1 clock cycle compared to the ones used in $(z-r)^2 / (2 \cdot \sigma'^2)$. Similarly, the computation of ccs does not lie on the critical datapath, and hence uses a higher latency **FPR_Mult**.

The optimized schedule, shown in Figure 1, employs selective FPU instantiation to improve timing closure. We compute terms x_1 and x_2 in parallel using duplicated **FPR_Mult** units with asymmetrical latencies: a faster 2-cycle multiplier for x_1 and a 3-cycle multiplier for x_2 (and ccs). This allows both terms to complete simultaneously at cycle 8 despite starting 2 cycles apart. While this parallelization requires 8 additional DSP slices, it reduces **SamplerZ** latency by 5 cycles. Given the total latency range of 22–35 cycles, this reduction significantly improves both **SamplerZ** and overall signature generation performance.

3.1.3 Speed-optimized flooring circuit for floating-point inputs

We propose a speed-optimized floating-point flooring algorithm in Algorithm 5 and its corresponding circuit in Figure 3, integrated for the first time in the context of **SamplerZ**. This module is critical, as it is repeatedly used to compute both $\text{floor}(a)$ for a floating-point input a , and the corresponding fractional component $a - \text{floor}(a)$.

Computing the floor of a floating-point number is non-trivial in hardware, since standard

floating-point to integer conversions follow a rounding mode (e.g., round-to-nearest) that may not match the floor operation. Our `FPR_Floor` circuit explicitly computes both $\text{floor}(a)$ and the residual $a - \text{floor}(a)$ by evaluating two candidate results in parallel.

Algorithm 5 Proposed `FPR_Floor`(a)

Require: $a \in \mathbb{R}$ (floating-point number)

Ensure: $\text{Output} = (\text{floor}(a), a - \text{floor}(a))$

```

1:  $r \leftarrow \text{FPR2Int}(a)$  ▷ Convert  $a$  to integer via rounding
2:  $a_1 \leftarrow \text{Int2FPR}(r)$ ;  $a_2 \leftarrow \text{Int2FPR}(r - 1)$ 
3:  $b_1 \leftarrow a - a_1$ ;  $b_2 \leftarrow a - a_2$  ▷ Subtraction using FPR_Sub
4: if  $b_1[63] = 0$  then ▷ Check the sign bit of  $b_1$ 
5:   return  $r, b_1$ 
6: else
7:   return  $r - 1, b_2$ 
8: end if

```

This parallelized design avoids the serial dependency of typical floor implementations that use a single conversion path (e.g., `FPR2Int` $\rightarrow$ `Int2FPR` $\rightarrow$ `FPR_Sub`). Our design reduces the critical path latency by 3 clock cycles compared to the state-of-the-art design by Ouyang *et al.* [OZZ⁺24], at a marginal cost of 3 additional DSPs.

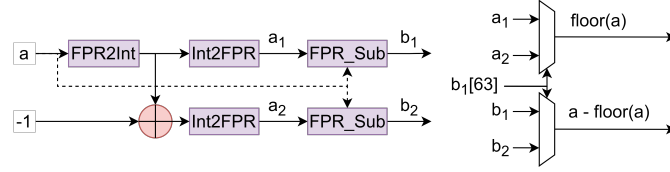


Figure 3: Proposed `FPR_Floor` circuit to calculate both the integer part and the fractional part of an input floating-point number.

3.1.4 Quantitative analysis on optimizing the number of sampling loops for `SamplerZ`

Lee *et al.* [LYN⁺24] present detailed statistics on `SamplerZ` loop executions over 102,400 test runs in Table 1. We replicated these experiments on our own setup and confirmed their findings. However, while these statistics help estimate the rejection probability of a single sampling loop, they do not accurately model the behavior of parallel hardware architectures.

Both our design and those in prior work employ two `SamplerZ` modules running in parallel to handle the independent invocations required for signature generation. Crucially, the signature generation process cannot proceed until both instances complete. Table 2 accounts for this synchronization requirement. It presents the failure distribution (i.e. the number of rejected samples before an accepted sample) specifically for the case of two `SamplerZ` modules running in parallel.

Table 1: Execution statistics of one `SamplerZ` hardware block. [LYN⁺24]

# Sampling loop iterations	1	2	3	4	5	≥ 6	Total
# <code>SamplerZ</code> executions	57580	24425	10361	4395	1865	1374	100000

We use our empirical data (Table 2) to estimate the effectiveness of parallel sampling loops, as shown in Figure 4a. While increasing the number of loops accelerates convergence, resource utilization scales linearly. For instance, a 5-loop design achieves a 97.2% success probability in a single iteration but consumes 4.4 $\times$ the resources of a single loop. In

Table 2: Execution statistics of two parallel **SamplerZ** hardware blocks.

# Sampling loop iterations	1	2	3	4	5	6	7	8	9	≥ 10	Total
# SamplerZ executions	33155	22162	14814	9903	6620	4425	2958	1977	1322	2665	100000

contrast, the 3-loop configuration offers a more practical trade-off, achieving 97.8% success within two iterations while requiring only $2.7\times$ the resources. The 2-loop configuration provides a balanced baseline, reaching 89.3% success in two iterations and converging to 99.6% within five.

We introduce **Rejection-Adjusted Execution Time (RAET)** to quantify the average execution time of **SamplerZ** during signature generation, accounting for the rejection probabilities in Table 2. Figure 4b presents the RAET normalized against the execution time of a single non-rejected instance of **SamplerZ**. For a design with i parallel loops and base latency t_i (without rejection), the RAET scales based on the convergence rate. Specifically, a 5-loop design achieves an RAET of $1.03 \times t_5$, while a 3-loop design yields $1.17 \times t_3$. This indicates that the 3-loop configuration offers comparable performance to the 5-loop case but with significantly lower resource utilization.

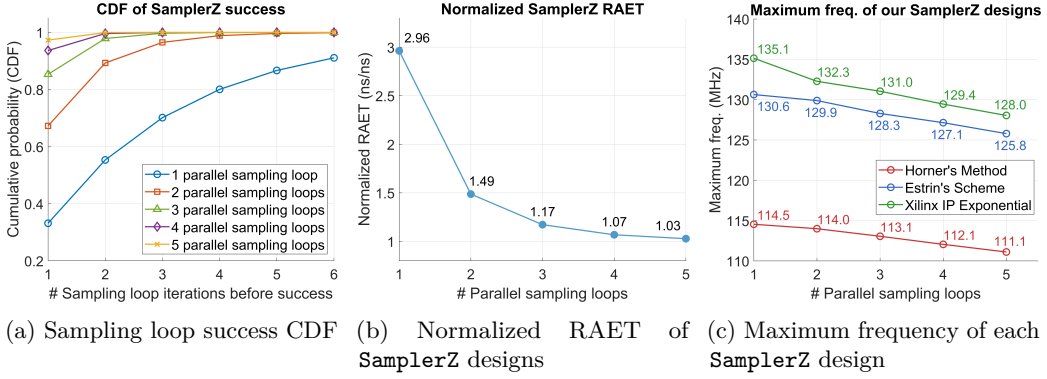


Figure 4: **SamplerZ** sampling loop rejection ratio based quantitative analysis for optimizing the number of parallel sampling loops in 2 parallel **SamplerZ** modules.

Figure 4c illustrates the maximum clock frequency for varying parallel sampling loops. As expected, higher loop counts increase resource utilization, causing Vivado to achieve lower maximum frequencies. Thus, the base latency t_i increases with the number of loops i ($t_i > t_j \forall i > j$). Consequently, while moving from 3 to 4 loops improves RAET by 7%, moving from 4 to 5 loops yields only a 2.5% gain. These marginal improvements come at the cost of significantly higher resource usage. This contrasts with Lee et al. [LYN⁺24], who advocated for 4 parallel loops; our analysis shows that the frequency penalty from increased utilization diminishes the returns of adding loops beyond 3.

Unlike Lee *et al.* [LYN⁺24], which instantiates 4 full **SamplerZ** blocks to achieve throughput improvements, our approach focuses on duplicating only the Sampling Loop logic, which directly impacts RAET, while sharing the control and Pre-Sampling logic across instances. This allows our multi-loop design to avoid repeated computation in **SamplerZ**'s Pre-Sampler phase and reduces resource utilization, which is particularly important in resource-constrained FPGA designs.

In summary, the most significant RAET reduction occurs when moving from 1 to 2 parallel loops (50% improvement), followed by a 20% gain from 2 to 3 loops. Beyond this, returns diminish, with only 7% and 2.5% improvements for 4 and 5 loops, respectively. While single-loop designs are the most resource-efficient, the 3-loop configuration offers near-optimal RAET at $2.7\times$ the resource cost. The 2-loop design presents a compelling middle

ground: it achieves a normalized RAET of $1.49 \times t_2$ (effectively doubling performance) while requiring only $1.8\times$ the resources (80% more DSPs). These findings enable designers to tailor the architecture to specific performance or resource constraints.

3.1.5 Circuit-level optimization of other modules

We fully unroll the **BerExp** comparison loop, which is responsible for comparing a uniformly sampled random number against a computed threshold. Specifically, lines 5 to 8 of Algorithm 4 perform a byte-wise comparison of the scaled output of **ApproxExp** against input uniform random bits. Prior implementations [OZZ⁺24, KA24, YSZ⁺24] execute this comparison sequentially over eight clock cycles. In contrast, our design parallelizes the operation by performing comparisons against all output bytes of **ApproxExp** simultaneously. This optimization reduces the overall latency of **SamplerZ** by 8 cycles in FALCON-512 and 16 cycles in FALCON-1024, while incurring minimal overhead of fewer than 100 flip-flops (FF) and 150 look-up tables (LUTs).

The **BaseSampler** design, as described by Algorithm 2, is a straightforward implementation in hardware. We perform all comparisons of $RCDT[i]$ against the input random bytes in parallel using combinatorial logic, and add the relevant bits in a single clock cycle, resulting in a latency of 1 clock cycle. Our **BaseSampler** can operate at 300MHz.

We configure the blocking Xilinx floating-point IPs, **FPR_Mult** and **FPR_Sub**, for low latencies of 2 and 1 clock cycles, respectively. While higher-latency, non-blocking configurations can increase maximum frequency, they disproportionately increase the latency of the design. For instance, a 3-cycle **FPR_Mult** improved frequency by 10% but added 5 cycles to **SamplerZ** latency, degrading overall execution time. Similarly, increasing **FPR_Sub** latency offered negligible frequency gains since **FPR_Mult** remained the critical path bottleneck. Unlike Ouyang *et al.* [OZZ⁺24], who utilized a 2-cycle subtractor for a high-frequency pipeline, we found the 1-cycle configuration optimal for our low-latency architecture, balancing clock speed with minimal execution time. This discrepancy is likely due to differences in architectural context: their design targets a high-frequency, high-latency pipeline, while ours is optimized for minimal execution time, which is based on finding a balance between high-frequency and low-latency.

3.2 Algorithmic innovations

3.2.1 Hardware-friendly high-throughput polynomial evaluation using Estrin’s Scheme

To maximize throughput of the exponential function required for **ApproxExp**, we propose the first replacement of Horner’s Method with Estrin’s Scheme for polynomial evaluation in the context of FALCON. While the FALCON specification follows Horner-style evaluation [FHK⁺18], it can be replaced by another method (shown in Equation 5) that is functionally equivalent. This approach allows multiple subexpressions to be evaluated concurrently and achieves a logarithmic evaluation depth $\mathcal{O}(\log n)$ under ideal parallelism. Intermediate powers of x (e.g., $x^2, x^4, \dots$) are computed recursively and reused across grouped terms. Estrin’s Scheme is particularly well-suited for architectures with deep pipelines or parallel multiply-accumulate units, such as FPGA DSP slices or SIMD processors.

$$P(x) = (a_0 + a_1x) + (a_2 + a_3x)x^2 + (a_4 + a_5x)x^4 + \dots \quad (5)$$

The Estrin’s Scheme implementation of **ApproxExp** can be seen in Algorithm 6, and the block design of the Estrin’s Scheme and Horner’s Method implementation can be seen in Figure 5. The **AE_Loop** block in the design corresponds to lines 4–6 of the **ApproxExp** algorithm using Horner’s Method (Algorithm 3) and lines 3–16 of the implementation using Estrin’s Scheme (Algorithm 6). Our implementation adopts a **fully parallelized**

version of Estrin’s Scheme, where multiple polynomial terms and powers of x are evaluated simultaneously. This allows the latency of the **ApproxExp_Loop** to be reduced from 12 clock cycles (in Horner’s Method) to 5 clock cycles. To prevent the power computation stage (line 3 in Algorithm 6) from becoming the new critical path, we introduce an additional pipeline stage immediately after power generation. This pipelining improves the maximum clock frequency of the **ApproxExp** module by 11%. While this fully parallelized Estrin-based implementation reduces the latency of **ApproxExp** by 44% and **SamplerZ** by 30%, it comes at the cost of 80 additional DSPs compared to the pipelined Horner’s Method version. This tradeoff is specific to our parallelized instantiation, which utilizes 6 parallel MAC units. Our design choice was dictated by our goal to reduce RAET of **SamplerZ**. In principle, Estrin’s Scheme can also be serialized to reduce resource usage, albeit with increased latency.

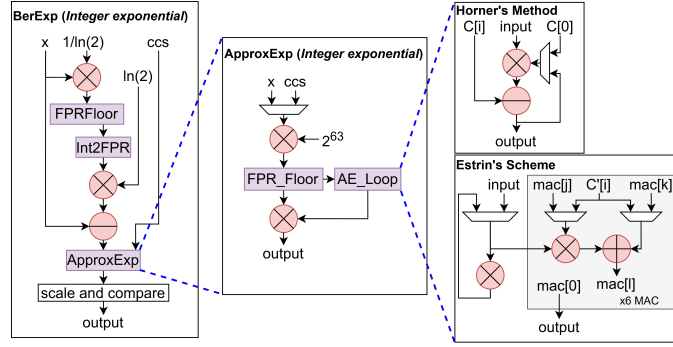


Figure 5: Expanded block diagram of integer domain **BerExp**, and two methods of evaluating **ApproxExp** (Estrin’s Scheme and Horner’s Method).

Algorithm 6 APPROXEXP_ESTRIN($x, ccs, \mathbf{C}$)

Require: Real input $x \in [0, \ln 2]$, constant $ccs \in [0, 1]$, polynomial coefficients $\mathbf{C} = [C_0, C_1, \dots, C_{12}]$ in Q0.63 fixed-point.

Ensure: Integer approximation of $2^{63} \cdot ccs \cdot \exp(-x)$ in Q0.63 domain.

- 1: $X \leftarrow \lfloor x \cdot 2^{63} \rfloor$
 - 2: $Z \leftarrow \lfloor ccs \cdot 2^{63} \rfloor$
 - 3: $x^2 \leftarrow (x \cdot x) \ggg 63$
 - 4: **for** $i \leftarrow 0$ to 5 **do** $\triangleright$ Estrin Stage 1: Perform 6 MAC in parallel
 - 5: $T_i \leftarrow C_{2 \cdot i} - ((C_{(2 \cdot i) + 1} \cdot x) \ggg 63)$
 - 6: **end for**
 - 7: $T_6 \leftarrow C_{12}$
 - 8: **for** $i \leftarrow 0$ to 2 **do** $\triangleright$ Estrin Stage 2: Perform 3 MAC in parallel
 - 9: $P_i \leftarrow T_{2 \cdot i} - ((T_{(2 \cdot i) + 1} \cdot x^2) \ggg 63)$
 - 10: **end for**
 - 11: $P_3 \leftarrow T_6$
 - 12: $x^4 \leftarrow (x^2 \cdot x^2) \ggg 63$
 - 13: $Q_0 \leftarrow P_0 + ((P_1 \cdot x^4) \ggg 63)$ $\triangleright$ Estrin Stage 3: Perform 2 MAC in parallel
 - 14: $Q_1 \leftarrow P_2 + ((P_3 \cdot x^4) \ggg 63)$
 - 15: $x^8 \leftarrow (x^4 \cdot x^4) \ggg 63$
 - 16: $R \leftarrow Q_0 + ((Q_1 \cdot x^8) \ggg 63) + 1$ $\triangleright$ Estrin Final Stage
 - 17: **return** $(R \cdot Z) \ggg 63$
-

3.2.2 Constant-latency BerExp via floating-point exponentiation

We propose a new instantiation of BerExp leveraging Xilinx IPs in place of ApproxExp in Algorithm 7. This approach significantly reduces both the latency and the resource utilization associated with the BerExp operation, while ensuring constant-time execution of the exponentiation, which is a critical requirement for preventing timing-based side-channel attacks. By performing exponentiation directly in the floating-point domain using FPR_Exp, we eliminate several intermediate operations required in the original integer-domain implementation, specifically those corresponding to lines 1-2 of Algorithm 4, which decompose the input x into a scaled form suitable for fixed-point exponential approximation, and line 3 of Algorithm 3, thereby removing additional datapath logic from the critical path. Block diagrams of both the integer domain and the floating-point domain exponentiation BerExp are shown in Figure 6.

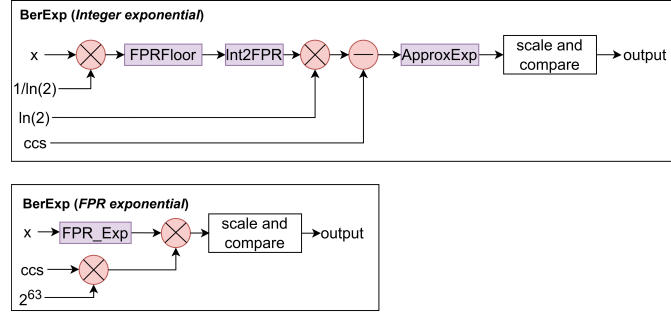


Figure 6: Expanded block diagram comparing integer domain and floating-point representation exponential BerExp.

Algorithm 7 Optimized BerExp(x, ccs)

Require: $x, ccs \geq 0$

Ensure: Boolean $b \in \{0, 1\}$ with $\Pr[b = 1] = ccs \times \exp(-x)$

- 1: $\tilde{y} \leftarrow \text{Exp}(-x) \times ccs$ ▷ Use Xilinx IP for exponentiation
 - 2: $Z \leftarrow \lfloor \tilde{y} \times 2^{64} - 1 \rfloor$ ▷ Convert to 64-bit Fixed Point for comparison
 - 3: $i \leftarrow 64 - k$
 - 4: **repeat** ▷ Note: this loop is fully unrolled (8x) for speedup.
 - 5: $u \leftarrow \text{UniformBits}(8)$
 - 6: $w \leftarrow u - ((Z \gg (i - 8)) \& 0xFF)$
 - 7: $i \leftarrow i - 8$
 - 8: **until** $(w \neq 0) \vee (i \leq 0)$
 - 9: **return** $(w < 0)$
-

4 Results

In this section, we present the implementation results of our FPGA designs. All proposed designs were written in SystemVerilog, targeting the Zynq Ultrascale+ XCZU7EV-FFVC1156-2-e FPGA. The designs were simulated using Questasim 2025.1 and synthesized and implemented on the FPGA using Xilinx Vivado 2024.2. We implement all architectural and circuit-level optimizations discussed in Section 3.1 in our designs. We evaluate our architectural innovations in three separate designs: Horner’s Method-based **SamplerZ** (version A), Estrin’s Scheme-based **SamplerZ** as discussed in Section 3.2.1 (version B), and Xilinx exponential IP-based **SamplerZ** as discussed in Section 3.2.2 (version C).

The remainder of this section is organized as follows: Section 4.1 provides the implementation results of our three **SamplerZ** designs and their submodules, Section 4.2

compares our FPGA designs against related FPGA designs that present **SamplerZ** results [OZZ⁺24, KA24, YSZ⁺24], Section 4.3 compares our best performing FPGA design against state-of-the-art ASIC and software works [OZZ⁺24, NG23], and Section 4.4 presents estimates for our designs when extended to a potential ASIC solution.

4.1 FPGA Implementation Results

Table 3 presents the FPGA results of the major submodules in our design. The double-precision FPUs **Int2FPR**, **FPR2Int**, **FPR_Mult**, **FPR_Sub**, and **FPR_Exp** are Xilinx floating-point IPs instantiated in blocking mode to perform operations including multiplication, subtraction, exponentiation, and conversions between double-precision floating-point and 64-bit integer domains. The latencies for the Xilinx floating-point IPs were optimized to minimize the RAET of our design. Specifically, extensive testing of **FPR_Exp** with latencies ranging from 6 to 10 cycles revealed that a blocking configuration with an 8-cycle latency yields the fastest **SamplerZ** performance. Similarly, we evaluated **FPR_Mult** and **FPR_Sub** with latencies from 1 to 3 cycles; the units located in the critical datapath were tuned for maximum speed based on these tests with a latency of 2 and 1 cycles, while their non-critical counterparts were set to latencies of 3 and 2 cycles, respectively. The reported LUT/FF utilization for these units represents the average across all instantiations within the various **SamplerZ** designs.

Table 3: FPGA implementation results of the major submodules in our **SamplerZ**

Submodules of SamplerZ		LUTs	FFs	DSPs	BRAMs	Latency (cycles)
Double precision floating-point IPs	Int2FPR	370	69	0	0	1
	FPR2Int	320	69	0	0	1
	FPR_Mult	94	189	8	0	2
	FPR_Sub	761	153	3	0	1
	FPR_Exp	693	575	26	2.5	8
BerExp	Version A	2874	2046	53	0	25
	Version B	3949	1860	139	0	18
	Version C	1272	1216	42	2.5	12
ApproxExp	Version A	937	656	34	0	16
	Version B	2200	793	120	0	9
BaseSampler		721	62	0	0	1
ChaCha20		5342	6204	0	0	N/A

Version A: Horner’s Method-based **SamplerZ**; Version B: Estrin’s Scheme-based **SamplerZ**; Version C: Xilinx exponential IP-based **SamplerZ**.

Table 4 presents the FPGA results of the three major architectural versions of our design, with each architecture including 1, 2, and 3 parallel sampling loop variants based on our prior analysis in Section 3.1.4. The execution time of these designs in the context of full signature generation is presented along with a comparison against related works in Section 4.2.

Horner’s Method-based **SamplerZ** is a direct implementation of the base algorithm for **FALCON**, showcasing improvements achieved solely through our architectural and circuit-level innovations. While the base latency is fixed at 35 cycles, parallelization significantly reduces effective sampling time: **A2** and **A3** achieve 50% and 60% reductions, respectively, compared to **A1**. Furthermore, parallel loops are more resource-efficient than module duplication. For instance, **A2** provides the functional equivalent of two **A1** instances using only $1.7\times$ LUTs and $1.9\times$ DSPs. Similarly, **A3** consumes roughly $2.2\times$ LUTs and $2.7\times$ DSPs. This resource scaling trend is consistent across all design versions (A, B, and C).

Estrin’s Scheme-based **SamplerZ** (Version B) leverages parallelized polynomial evaluation (Section 3.2.1) to reduce latency to 28 cycles, a 20% improvement over Version A. Combined with higher operating frequencies, this yields a 30% reduction in sampling

Table 4: FPGA implementation results of our **SamplerZ** designs

SamplerZ designs			Parallel sampl. loops	SamplerZ utilization (LUT/FF/DSP/BRAM)	SamplerZ latency (cycles)	$F_{\max}$ (MHz)	SamplerZ time ¹ (ns)	SamplerZ K-cycles ¹
Our SamplerZ designs (FPGA)	Horner’s Method	A1	1	10k/6k/108/0	35	115	463/927	53/106
		A2	2	17k/10k/202/0	35	114	234/467	27/53
		A3	3	22k/14k/293/0	35	113	186/372	21/42
	Estrin’s Scheme	B1	1	9k/5k/188/0	28	131	325/650	42/85
		B2	2	16k/8k/365/0	28	130	160/321	21/43
		B3	3	23k/11k/542/0	28	128	131/262	17/34
	FP-Exp. ²	C1	1	7k/4k/94/2.5	22	135	247/494	33/67
		C2	2	12k/7k/174/5	22	132	127/253	17/33
		C3	3	15k/9k/254/7.5	22	131	101/201	13/26

¹ For FALCON 512/1024.² Xilinx floating-point exponential IP-based design.

time for configurations B1–B3 compared to their Version A counterparts. However, this performance gain comes at a cost: the parallelized **ApproxExp** routine requires 80 additional DSPs per loop (a 74% increase), though it utilizes 15% fewer LUTs and FFs on average.

The Xilinx exponential IP-based SamplerZ (Version C) utilizes our **BerExp** algorithm (Section 3.2.2) to perform exponential evaluation directly in floating-point. By eliminating conversion between floating-point and integer domains in **BerExp**, this design reduces resource usage by 27% in LUTs/FFs and 13% in DSPs (14 fewer) compared to Version A. Furthermore, it achieves a latency of just 22 cycles, resulting in sampling time reductions of 47% and 24% over Versions A and B, respectively.

4.2 Comparison against related FPGA works

This section compares our FPGA **SamplerZ** results with prior works [OZZ⁺24, YSZ⁺24, KA24], as summarized in Table 5, of which Ouyang *et al.* [OZZ⁺24] (TCHEs 2025) is the state-of-the-art baseline for high-performance FPGA designs. While most listed designs utilize a single sampling loop, our versions A3, B3, and C3 employ three parallel loops. Results for the 2-loop configurations (A2, B2, C2) are omitted for brevity but can be inferred from Table 4.

Hardware performance evaluation. For each hardware design, we use reported latency and clock frequency to adjust for rejection and compute RAET, as described in Section 3.1.4, yielding **SamplerZ** latency and execution time within full signature generation. Since **SamplerZ** execution time varies with input, RAET ensures fair comparison.

RAET of SamplerZ. Table 5 shows that the FPGA baseline [OZZ⁺24] reports a **SamplerZ** pair latency of 67–74 cycles (FPGA, 185 MHz), which corresponds to an RAET of 1073 ns. Our best designs (C1, C2, C3) achieve 22-cycle latency at 135 MHz, 132 MHz, and 131 MHz, yielding RAETs of 482 ns, 247 ns, and 197 ns, respectively. All of our FPGA designs have lower RAET than the FPGA baseline, with our best FPGA design outperforming the FPGA baseline by $2.2\times$ – $5.4\times$.

Sampling time in signature generation. In the context of full signature generation for FALCON-1024, the sampling time in the baseline FPGA is 1099 μ s. In comparison, our **SamplerZ** designs (Versions C1, C2, and C3) have a sampling time of 494 μ s, 253 μ s, and 201 μ s. All our FPGA designs outperform baseline FPGA, including our best performing versions (C1, C2, and C3), which outperform the baseline FPGA by 55%–82%.

Signature generation. In the context of full signature generation, our **SamplerZ** architecture reduces the impact of **SamplerZ** in signature generation. While **SamplerZ** accounts for 64% of the execution time of full signature generation in the FPGA baseline, our designs reduce this to under 6%. Consequently, FALCON-1024 signature generation

Table 5: Comparison of our FPGA **SamplerZ** designs in full FALCON-512/1024 Signature Generation against related FPGA works (SW: Software Design, HW: Hardware Design)

SamplerZ designs			SamplerZ utilization (LUT/FF/DSP/BRAM)	SamplerZ latency ¹ (cycles)	$F_{\max}$ (MHz)	SamplerZ RAET (ns)	SamplerZ in full signature gen.		Signature gen. time ² (μ s)
							time (μ s)	K-cycles	
HW/SW (FPGA) ³ [KA24]			4068/3202/9/2	1807	121	44.2k	23k/45k	187/374	208k/417k
HW/SW (FPGA) [YSZ ⁺ 24]			4761/692/9/0	104	83	3712	1.9k/3.8k	158/315	19k/38k
HW (FPGA) ⁴ [OZZ ⁺ 24] (FPGA Baseline)			9368/4527/76/0	67 ⁵	185	1073	549/1099	102/203	865/1716
Our SamplerZ designs (FPGA)	Horner's	A1	10k/6k/108/0	35	115	905	463/927	53/106	761/1528
		A3	22k/14k/293/0	35	113	363	186/372	21/42	484/973
		B1	9k/5k/188/0	28	131	635	325/650	42/85	623/1251
	Estrin's	B3	23k/11k/542/0	28	128	256	131/262	17/34	429/863
		C1	7k/4k/94/2.5	22	135	482	247/494	33/67	545/1095
	FP-Exp. ⁶	C3	15k/9k/254/7.5	22	131	197	101/201	13/26	399/802

¹ Latency of **SamplerZ** without rejection, except for [NG23], for which the observed latency (including rejection) is presented. Further calculations account for this.

² Estimated by plug-and-play of our **SamplerZ** in the FFSampling of [OZZ⁺24].

³ Performance-optimized design from [KA24] used for comparison.

⁴ Utilization of the ChaCha20 module from [OZZ⁺24] is subtracted for fair comparison against prior works [KA24, YSZ⁺24].

⁵ [OZZ⁺24] reported FPGA latency of 67 to 74 cycles for **SamplerZ**; 67 cycles is used here.

⁶ Xilinx floating-point exponential IP-based design.

time for our best performing version ranges from 1095 μ s (C1) to 802 μ s (C3), which is based on our **SamplerZ** designs and the FFSampling from the FPGA baseline [OZZ⁺24]. This represents a 36%-54% improvement in signature generation time over the FPGA baseline design.

Resource utilization. The **SamplerZ** module accounts for roughly 20% of the total resource utilization in LUT/FF/DSPs in FPGA baseline. Among our best designs, C1 offers a direct comparison to the FPGA baseline [OZZ⁺24], as both utilize a single sampling loop, whereas C2 and C3 employ 2 and 3 parallel sampling loops, respectively. The LUT/FF utilization of C1 is roughly 21%/10% lower than the baseline FPGA, and requires 18 additional DSPs, which corresponds to 1% increased DSP utilization on this FPGA. However, both C2 and C3 utilize more resources than the FPGA baseline due to multiple parallel sampling loops. Design C2 utilizes 26%/49% more LUT/FF, and 98 additional DSPs, C3 utilizes 64%/106% more LUT/FF, and 178 additional DSPs. Despite the increased resource utilization, the corresponding reductions in execution time confirm that our designs offer compelling trade-offs between performance and resource utilization.

4.3 Comparison against related ASIC and software works

This section compares the FPGA implementation results of our **SamplerZ** against state-of-the-art ASIC result from the design by Ouyang *et al.* [OZZ⁺24] on 28nm TSMC, and state-of-the-art software from the design by Nguyen *et al.* [NG23] on 5nm Apple M1 SoC, presented in Table 6. We present only our fastest FPGA design here for brevity.

Hardware performance evaluation. Since Ouyang *et al.* [OZZ⁺24] do not explicitly report ASIC latency, we infer the FALCON-1024 and **SamplerZ** execution times and cycles based on their reported FALCON-512 signature generation time and clock frequency.

Software performance evaluation. For the fastest software design [NG23], we measured the median latency of a **SamplerZ** pair over 10k signature generation runs using the authors' timing function (*kpc_get_thread_counters*). This measurement inher-

Table 6: Comparison of our FPGA **SamplerZ** design in full FALCON-512/1024 Signature Generation against related ASIC and software works

SamplerZ designs	SamplerZ utilization (LUT/FF/DSP/BRAM)	SamplerZ latency ¹ (cycles)	$F_{\max}$ (MHz)	SamplerZ RAET (ns)	SamplerZ in full signature gen.		Signature gen. time ² (μ s)
					time (μ s)	K-cycles	
SW (Apple M1) [NG23]	N/A	740/730	3200	231/228	118/234	379/747	138/276
HW (ASIC 28nm TSMC) [OZZ+24]	N/A	67 ³	530	236	121/241	105/206	190/377
Our SamplerZ HW (FPGA) (FP-Exp ⁴ Version C3)	15k/9k/254/7.5	22	131	197	101/201	13/26	399/802

¹ Latency of **SamplerZ** without rejection, except for [NG23], for which the observed latency (including rejection) is presented. Further calculations account for this.

² Estimated by plug-and-play of our **SamplerZ** in the FFSampling of [OZZ+24].

³ [OZZ+24] have not reported ASIC latency for **SamplerZ**. Their FPGA design has a latency of 67-74. The rest of the values for ASIC (RAET, **SamplerZ** time, **SamplerZ** cycles) are based on the reported signature generation execution time on ASIC instead of **SamplerZ** latency.

⁴ Xilinx floating-point exponential IP-based design.

ently accounts for rejections in **SamplerZ**. All other results are taken directly from their publication.

RAET of **SamplerZ.** Table 6 shows our fastest FPGA design achieves a 22-cycle latency at 131 MHz, yielding an RAET of 197 ns. In comparison, the fastest ASIC design [OZZ+24] achieves an RAET of 236 ns (530 MHz), while the fastest software design (3.2 GHz) [NG23] achieves an RAET of 231/228 ns for FALCON-512/1024. Despite the frequency and technology disadvantages of FPGAs, our design achieves RAET improvements of 16.5% and 13.6% over the state-of-the-art ASIC and software designs, respectively.

Sampling time and cycles in signature generation. For FALCON-1024 signature generation, the ASIC requires 206 K-cycles (241 μ s) for sampling, while the software baseline requires 747 K-cycles (234 μ s). In comparison, our best FPGA **SamplerZ** design completes in just 26 K-cycles (201 μ s). This represents a cycle count reduction of 7.9 $\times$ and 28.7 $\times$, and execution time reduction of 16.5% and 13.6%, respectively.

Signature generation. For full signature generation (FALCON-512/1024), the ASIC baseline achieves 190/377 μ s and the software baseline 138/276 μ s. Our FPGA design requires 399/802 μ s, representing a 2.1 $\times$ and 2.9 $\times$ slowdown. This gap is expected, given the technological disparity between our FPGA platform and the 28nm ASIC or 5nm Apple M1 baselines.

4.4 ASIC extension

Since we do not have the 28nm TSMC technology node of Ouyang *et al.* [OZZ+24] available, this section estimates our work on a generic 28nm ASIC platform to enable a more direct comparison against their state-of-the-art ASIC design, as well as the state-of-the-art software design on Apple M1 (5nm) [NG23]. Prior works in estimating ASIC performance from FPGA designs [KR07, WBR11] show ASIC designs, which are based on FPGA designs that utilize hard blocks such as DSP and BRAM, have a critical-path latency of 3 $\times$ to 4 $\times$ lower than their FPGA counterparts. This translates directly to 3 $\times$ to 4 $\times$ increase in maximum frequency of the ASIC design when compared to the FPGA design utilizing hard blocks. This matches the results from the state-of-the-art ASIC and FPGA designs by Ouyang *et al.* [OZZ+24], with their ASIC design having a clock frequency of 2.9 $\times$ that of their FPGA design. Since we use the same FPGA and speed grade, and we

expect our ASIC design to perform at least $2.5\times$ faster than our FPGA design, which includes over 10% margin. This result for our fastest design C3 is shown in Table 7. While comparing a 28nm ASIC to the Apple M1 (5nm) is not directly comparable due to process node differences, our design achieves over $2.9\times$ faster sampling and improves signature generation by 42% and 21% over the fastest ASIC and software designs, respectively. Finally, the portability of our FPGA architecture into ASIC platforms is discussed in Section 5.1

Table 7: Estimated ASIC results of our **SamplerZ** design for FALCON-512/1024

Designs	Freq (MHz)	Sampling cycles	Sampling time (μ s)	Signature generation time (μ s)
SW (Apple M1 5nm) [NG23]	3200	379k/747k	118/234	138/276
HW (ASIC 28nm TSMC) [OZZ+24]	530	105k/206k	121/241	190/377
Our design C3 (estimated for ASIC 28nm)	328	13k/26k	40/81	110/216

4.5 Summary of results

While the state-of-the-art FPGA and ASIC design by Ouyang *et al.* [OZZ+24] prioritizes throughput for the entire signature generation process, it does not fully optimize **SamplerZ** itself. In contrast, our work focuses on designing the fastest **SamplerZ**, thereby addressing the primary bottleneck by reducing the **SamplerZ** share of full signature generation execution time from 64% in Ouyang *et al.*’s design to under 6% in our designs.

Compared to Ouyang *et al.*’s FPGA design [OZZ+24], all our FPGA designs achieve faster sampling time and sampling cycles, with our best design versions (C1, C2, and C3) on FPGA reducing sampling time by 55% to 81% and sampling cycles by 67% to 87%, which corresponds to a 36% to 53% reduction in full signature generation time. Moreover, our **SamplerZ** design achieves the lowest reported latency to date across all platforms, measuring just 22 cycles. Furthermore, in terms of sampling time, our **SamplerZ** design C3 on FPGA is 16% faster than state-of-the-art ASIC design (28nm TSMC) [OZZ+24] and state-of-the-art software design on high-performance CPU (Apple M1) [NG23], representing the fastest **SamplerZ** design to date. Extending our **SamplerZ** design to ASIC can reduce sampling time by over $2.9\times$ and increase signature generation by 42% and 21% compared to baseline ASIC and software, respectively.

5 Discussion

5.1 Transferability to other FPGAs and ASICs

Our proposed architectural optimizations are fundamentally built upon the Xilinx floating-point IPs, which means they are specifically designed to leverage the features and capabilities of Xilinx FPGAs. While it is possible to adopt a similar approach on Intel FPGAs by utilizing their native exponential IP, it is important to note that certain Intel IPs do not support variable latency. This difference would inevitably lead to distinct performance outcomes compared to the Xilinx FPGA based designs. Although we anticipate that a speedup would still be achieved on Intel FPGAs, we have not quantified this in our current work, as most existing research in this area has concentrated on Xilinx devices [OZZ+24, YSZ+24, SAW+23, KA24, CYZ+25, BPV25, PdRB+25, MBM25].

Furthermore, all of our proposed designs (versions A, B, and C) make use of Xilinx floating-point IP cores with AXI-S interfaces. While it is technically feasible to substitute these vendor-specific IPs with generic RTL implementations in order to improve portability

across different hardware platforms, our primary goal in this work was to achieve the highest possible performance, which requires taking advantage of such specialized, vendor-optimized components. In the context of ASIC implementations, these IPs would be replaced by custom RTL modules tailored to the specific requirements of the target application. Our preliminary review of available resources suggests that there are several open-source custom RTL modules for floating-point operations [FCF23, FL16, dDP11]. However, these alternatives would need to be carefully evaluated to ensure they are suitable for the low-latency and blocking configurations required by our architecture.

5.2 Reproducibility

Reproducibility is essential for validating claims, enabling fair comparisons, and advancing the state of the art in cryptographic hardware. Given the security-critical nature of post-quantum schemes and the architectural complexity of FALCON’s sampling subroutine, transparent and reproducible research is especially important.

We evaluated the FalconSign implementation from Ouyang *et al.* [OZZ⁺24], which provides an open-source repository with design files and documentation. While the available RTL enabled partial verification of their design methodology, we were unable to replicate the reported performance numbers—particularly with respect to timing and resource utilization. After reaching out to the authors, we were informed that the specific RTL version used to obtain the published results will not be publicly released, as it contained proprietary optimizations that were considered confidential intellectual property.

In contrast, to promote openness and verifiability, we will release our complete **SamplerZ** hardware designs publicly upon publication. The release will include all three exponential evaluation variants (Horner’s method, Estrin’s scheme, and floating-point IP configurations). This release will support reproducibility, enable fair benchmarking, and foster continued research into high-performance hardware implementations of post-quantum cryptographic primitives.

6 Conclusion

To the best of our knowledge, our **SamplerZ** design for FALCON achieves the lowest execution time and latency reported to date. Notably, this work presents the **first** hardware implementation of **SamplerZ** to surpass the performance of state-of-the-art software. We introduce novel architectural and algorithmic optimizations, including a constant-latency floating-point **BerExp** and a parallelized Estrin’s Scheme-based **ApproxExp**. These contributions advance FALCON hardware acceleration by demonstrating that aggressive latency reduction and sampling loop parallelization in **SamplerZ** yield substantial gains in sampling and signature generation time in FALCON. These results establish a new performance baseline for FALCON hardware implementations and enable practical deployment of reconfigurable devices in high-demand scenarios such as secure vehicle networks and digital payment systems. Future work will extend support to key generation, verification, and integrate hardened protections against implementation attacks.

References

- [Ajt96] M. Ajtai. Generating hard instances of lattice problems (extended abstract). In *Proceedings of the twenty-eighth annual ACM symposium on Theory of Computing*, STOC ’96, pages 99–108, New York, NY, USA, July 1996. Association for Computing Machinery.
- [BDK⁺22] J. P. B. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. Schanck, P. Schwabe, G. Seiler, and D. Stehlé. CRYSTALS-Dilithium: Algorithm

- Specifications and Supporting Documentation (Round 3 Submission). Available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>, 2022. Accessed: 2024-09-22.
- [BHH⁺22] D. Bernstein, D. Hopwood, A. Hülsing, L. Maram, T. Pöppelmann, P. Schwabe, G. Van Assche, and Z. Wilcox-O’Hearn. SPHINCS+: Submission to the NIST Post-Quantum Cryptography Standardization Project (Round 3). Available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>, 2022. Accessed: 2024-09-22.
- [BPV25] Jonas Bertels, Hilder V. L. Pereira, and Ingrid Verbauwhede. Final bootstrap acceleration on fpga using dsp-free constant-multiplier ntts. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(3):293–316, Jun. 2025.
- [CSD17] Information Technology Laboratory Computer Security Division. Post-Quantum Cryptography | CSRC | CSRC, January 2017.
- [CYZ⁺25] Cheng Chen, Gangqiang Yang, Hongchao Zhou, Hailiang Xiong, Xianye Ben, and Zhiguo Wan. Fusionmsm: A collision-free and arithmetic-optimized fpga-based accelerator for multi-scalar multiplication. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):1077–1101, Sep. 2025.
- [dDP11] Florent de Dinechin and Bogdan Pasca. Designing custom arithmetic data paths with flopoco. *IEEE Design Test of Computers*, 28(4):18–27, 2011.
- [DH76] W. Diffie and M. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, November 1976. Conference Name: IEEE Transactions on Information Theory.
- [DP16] Leo Ducas and Thomas Prest. Fast Fourier Orthogonalization. In *Proceedings of the 2016 ACM International Symposium on Symbolic and Algebraic Computation*, ISSAC ’16, pages 191–198, New York, NY, USA, July 2016. Association for Computing Machinery.
- [Est60] Gerald Estrin. Organization of computer systems: The fixed plus variable structure computer. In *Proceedings of the Western Joint Computer Conference*, pages 33–40, 1960.
- [FCF23] Michele Fiorito, Serena Curzel, and Fabrizio Ferrandi. Truefloat: A templatized arithmetic library for hls floating-point operators. In *International Conference on Embedded Computer Systems*, pages 486–493. Springer, 2023.
- [FHK⁺18] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang, et al. Falcon: Fast-Fourier lattice-based compact signatures over NTRU. *Submission to the NIST’s post-quantum cryptography standardization process*, 36(5):1–75, 2018.
- [FL16] Xin Fang and Miriam Leeser. Open-source variable-precision floating-point library for major commercial fpgas. *ACM Trans. Reconfigurable Technol. Syst.*, 9(3), July 2016.
- [HKR⁺18] John Howe, Amir Khalid, Chris Rafferty, Francesco Regazzoni, and Maire O’Neill. On practical discrete gaussian samplers for lattice-based cryptography. *IEEE Transactions on Computers*, 67(3):322–334, March 2018.

- [Hor20] William George Horner. A new method of solving numerical equations of all orders, by continuous approximation. *Philosophical Transactions of the Royal Society of London*, 109:308–335, 1820.
- [HPS98] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU: A ring-based public key cryptosystem. In Joe P. Buhler, editor, *Algorithmic Number Theory*, pages 267–288, Berlin, Heidelberg, 1998. Springer.
- [KA24] Emre Karabulut and Aydin Aysu. A Hardware-Software Co-Design for the Discrete Gaussian Sampling of FALCON Digital Signature. *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 90–100, May 2024. Conference Name: 2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST) ISBN: 9798350373943 Place: Tysons Corner, VA, USA Publisher: IEEE.
- [KLL21] Liang Kong, Shuguo Li, and Ruirui Liu. High-performance constant-time discrete gaussian sampling. *IEEE Transactions on Computers*, 70(7):1019–1033, July 2021.
- [KR07] Ian Kuon and Jonathan Rose. Measuring the gap between fpgas and asics. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 26(2):203–215, 2007.
- [KSS22] Youngbeom Kim, Jingyo Song, and Seog Chung Seo. Accelerating Falcon on ARMv8. *IEEE Access*, 10:44446–44460, 2022. Conference Name: IEEE Access.
- [LYN⁺24] Yongseok Lee, Jonghee Youn, Kevin Nam, Heon Hui Jung, Myunghyun Cho, Jimyung Na, Jong-Yeon Park, Seungsu Jeon, Bo Gyeong Kang, Hyunyoung Oh, and Yunheung Paek. An Efficient Hardware/Software Co-Design for FALCON on Low-End Embedded Systems. *IEEE Access*, 12:57947–57958, 2024. Conference Name: IEEE Access.
- [MBM25] Konstantina Miteloudi, Lejla Batina, and Nele Mentens. A5/3 make or break: A massively parallel fpga architecture for exhaustive key search. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(3):361–388, Jun. 2025.
- [Nat15] National Institute of Standards and Technology (NIST). SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Technical Report FIPS PUB 202, U.S. Department of Commerce, August 2015.
- [NG23] Duc Tri Nguyen and Kris Gaj. Fast falcon signature generation and verification using armv8 neon instructions. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *Progress in Cryptology - AFRICACRYPT 2023*, pages 417–441, Cham, 2023. Springer Nature Switzerland.
- [OZZ⁺24] Yi Ouyang, Yihong Zhu, Wenping Zhu, Bohan Yang, Zirui Zhang, Hanning Wang, Qichao Tao, Min Zhu, Shaojun Wei, and Leibo Liu. FalconSign: An Efficient and High-Throughput Hardware Architecture for Falcon Signature Generation. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):203–226, Dec. 2024.
- [PdRB⁺25] Xander Pottier, Thomas de Ruijter, Jonas Bertels, Wouter Legiest, Michiel Van Beirendonck, and Ingrid Verbauwhede. Optimism: Fpga hardware accelerator for zero-knowledge msm. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):489–510, Mar. 2025.

- [Per25] Ray Perlner. Fips 206 status update: FALCON (falcon). Presentation, 6th PQC Standardization Conference, 2025. National Institute of Standards and Technology (NIST).
- [QA25] Jinyi Qiu and Aydin Aysu. SHIFT SNARE: Uncovering secret keys in FALCON via single-trace analysis. Cryptology ePrint Archive, Paper 2025/146, 2025.
- [RSA78] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, February 1978.
- [SAW⁺23] Michael Schmid, Dorian Amiet, Jan Wendler, P. Zbinden, and Tao Wei. Falcon Takes Off - A Hardware Implementation of the Falcon Signature Scheme. *IACR Cryptol. ePrint Arch.*, 2023.
- [SBN⁺20] Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marc Manzano, and Ramesh Karri. *Hardware Architectures for Post-Quantum Digital Signature Schemes*. Springer International Publishing, 2020.
- [Sho97] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997. Publisher: Society for Industrial and Applied Mathematics.
- [WBR11] Henry Wong, Vaughn Betz, and Julie Rose. Comparing fpga vs. custom cmos and the impact on processor microarchitecture. pages 5–14, 02 2011.
- [YSZ⁺24] Xinglong Yu, Yi Sun, Yifan Zhao, Honglin Kuang, and Jun Han. RVCE-FAL: A RISC-V Scalar-Vector Custom Extension for Faster FALCON Digital Signature. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6, March 2024. ISSN: 1558-1101.
- [ZSL20] Raymond K. Zhao, Ron Steinfeld, and Dongxi Liu. FACCT: Fast, Compact, and Constant-Time discrete gaussian sampler over integers. *IEEE Transactions on Computers*, 69(1):126–137, January 2020.